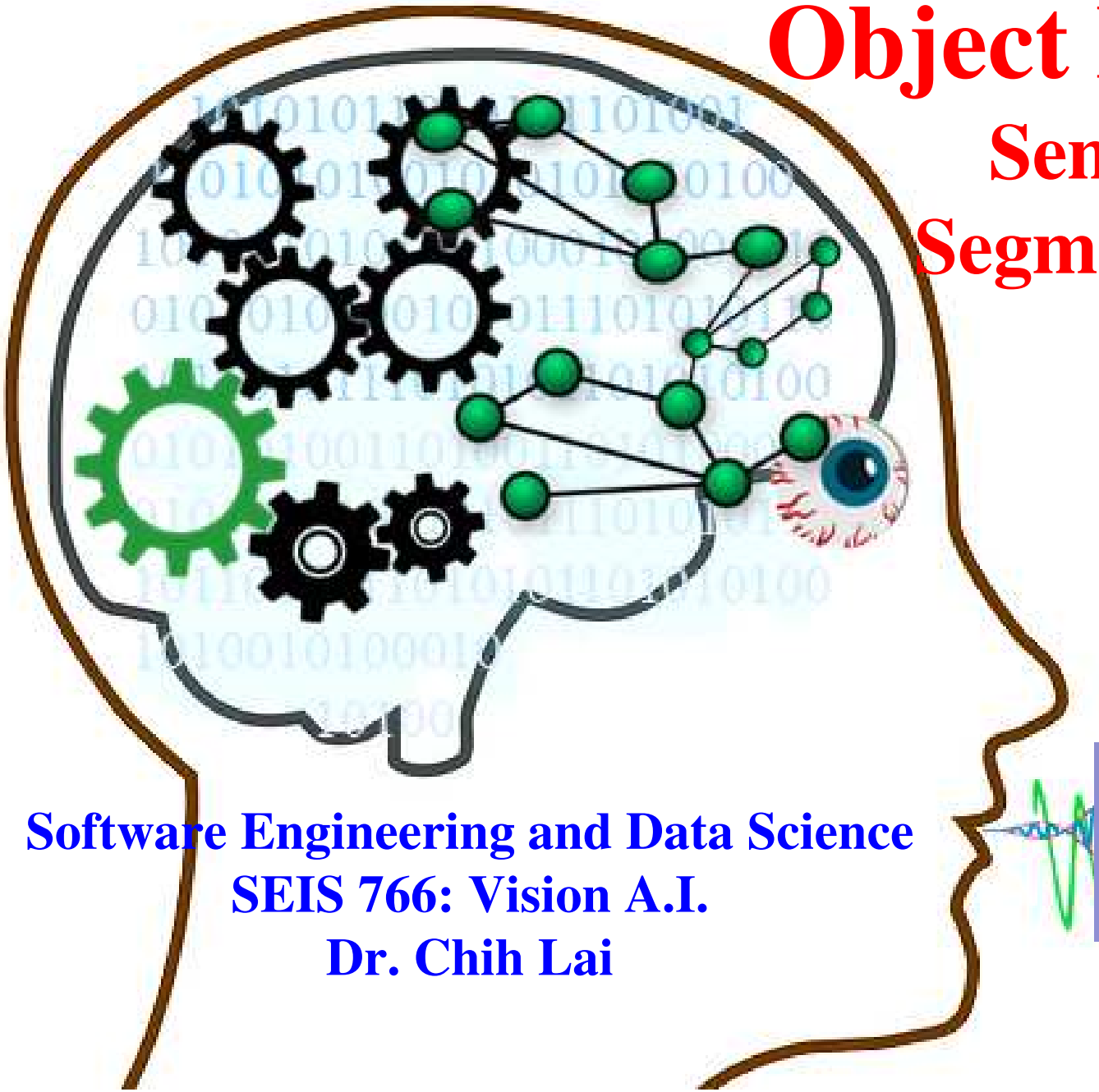
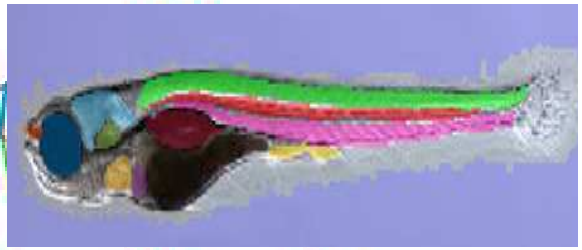
Object Detection
Semantic Segmentation

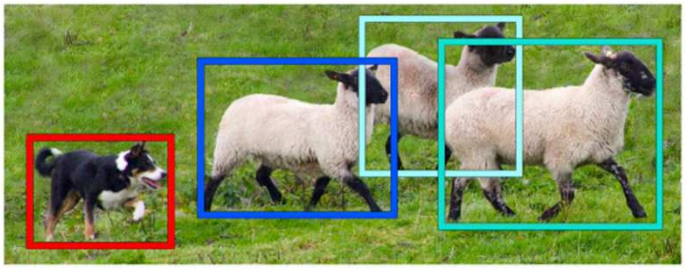


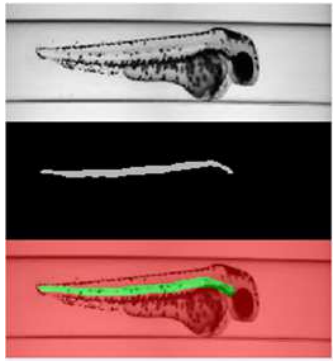


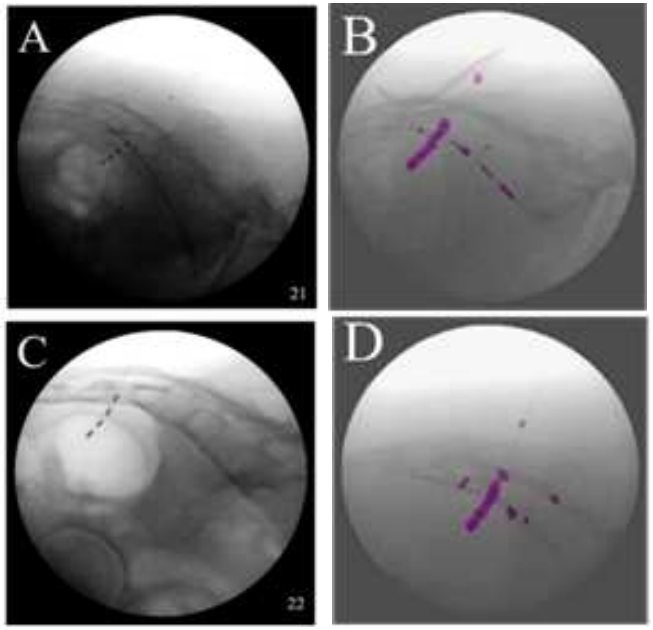
Software Engineering and Data Science
SEIS 766: Vision A.I.
Dr. Chih Lai

Outline

- Object Detection, Fast RCNN
- √ Object Detection, Semantic Segmentation, Labeling, Applications
- Encode + Decode CNN, Transposed Convolution, Pre-Trained CNN
- Quality Evaluation of Object Detection
- Identifying Morphological Changes in Organs w/ Occlusion Map



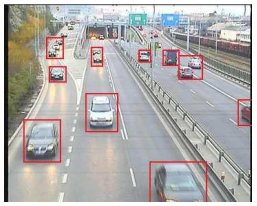


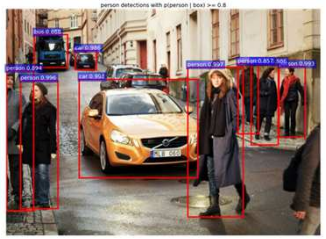


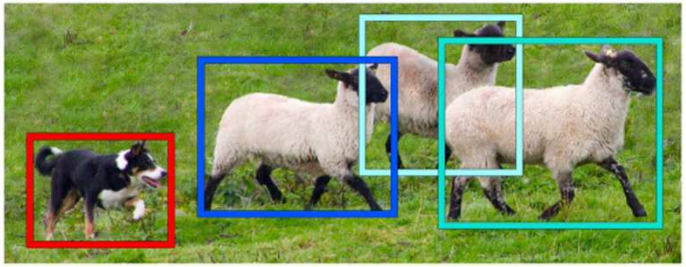
https://youtu.be/hxczTe9U8jY

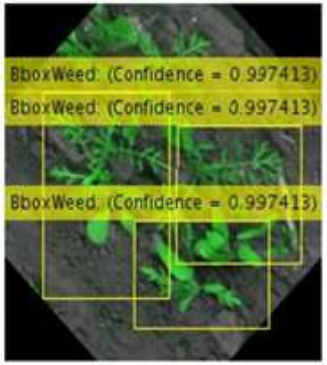
Different Types of Object Allocation

Image Recognition



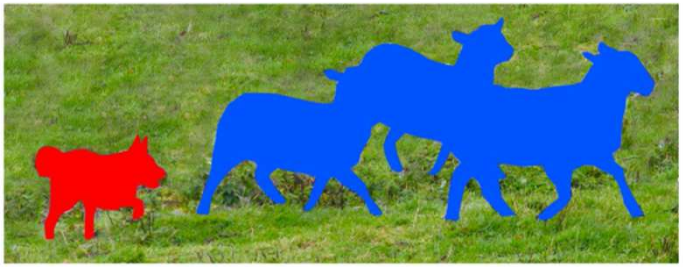




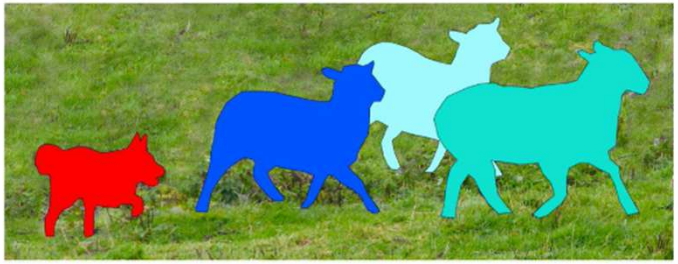


- RCNN
- Fast RCNN
- Faster RCNN
- Yolo
- SSD

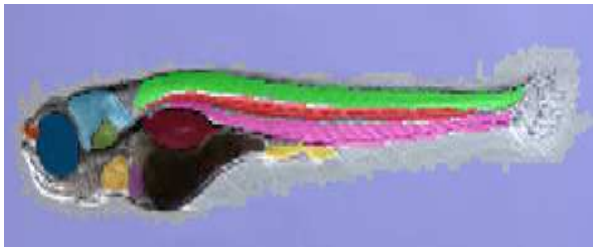
Object Detection

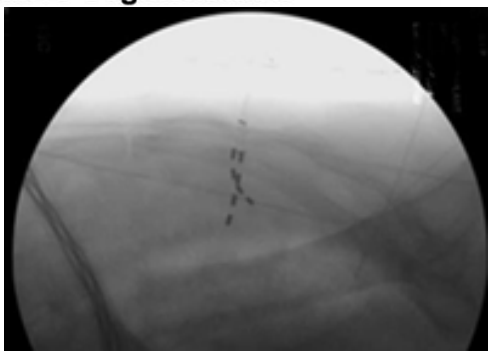


Semantic Segmentation



Instance Segmentation





Semantic Segmentation

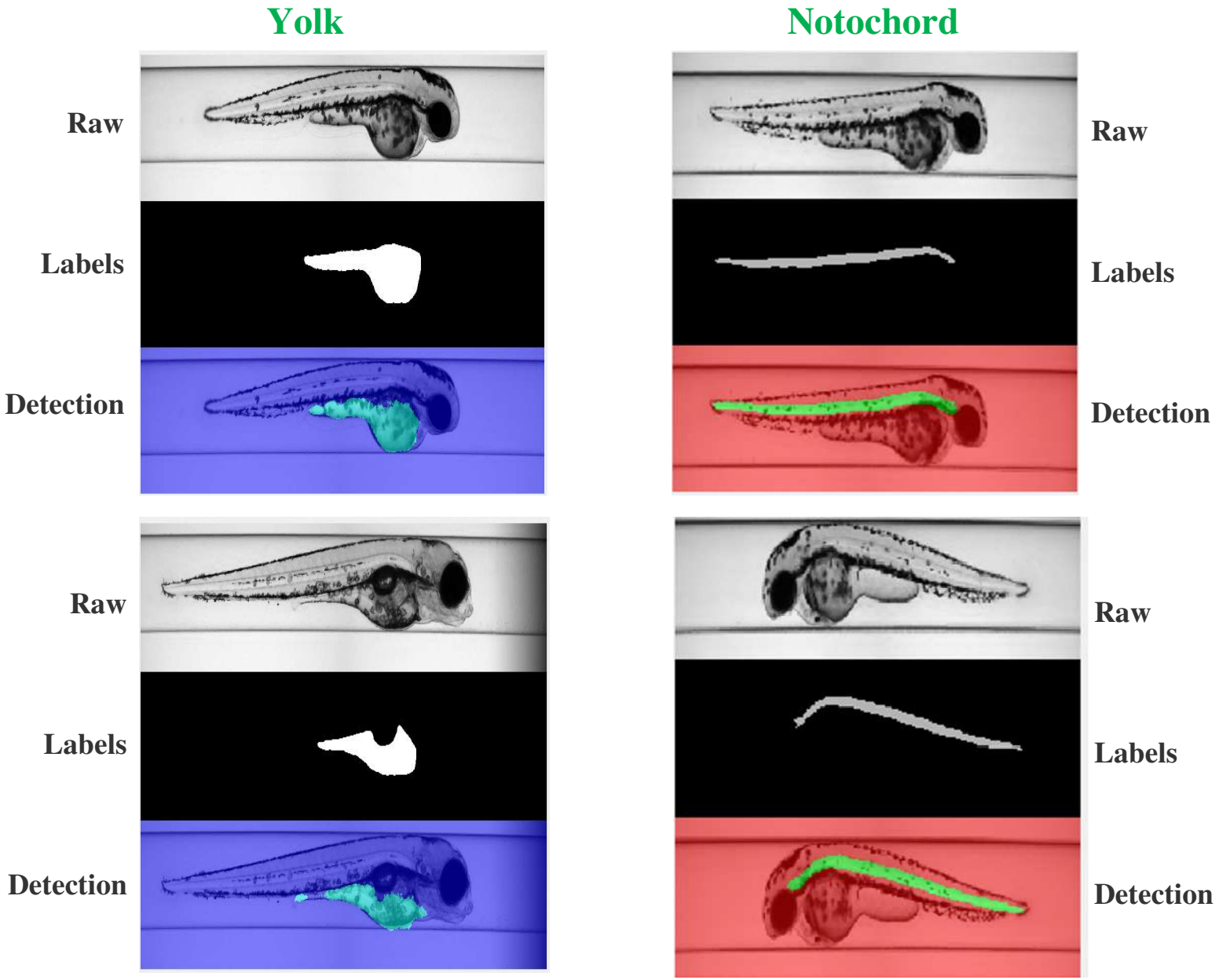
- Semantic segmentation classifies **EACH** pixel into a ***class*** (i.e. ***semantics***)
  - more precise way of object-identification (for AI surgery, precision agriculture, ID cancer cell, etc.)
  - But, higher computation demand
  - Early publication➔ https://arxiv.org/pdf/1511.00561.pdf

NO car1, car2, ……

https://www.mathworks.com/help/vision/examples/semantic-segmentation-using-deep-learning.html

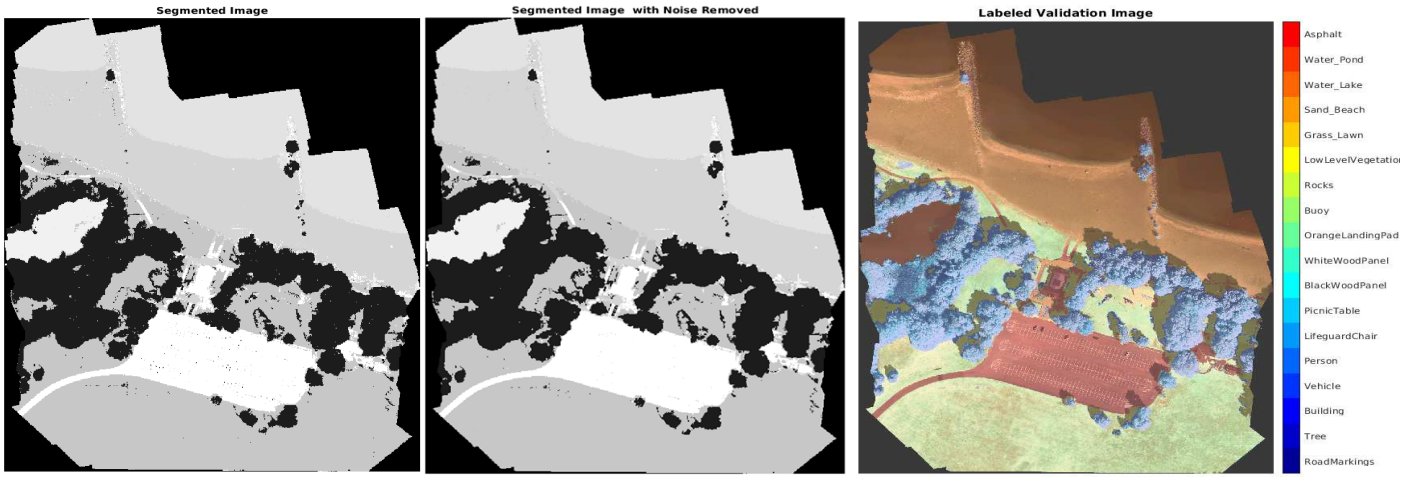
https://www.mathworks.com/help/vision/ug/semantic-segmentation-examples.html#mw_6ab02754-d2fa-4330-8bea-3eeec77279da

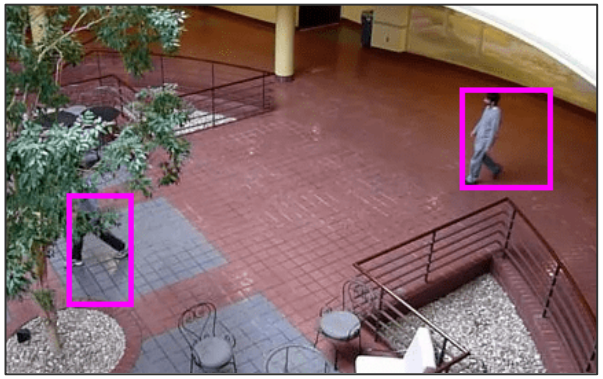
Semantic Segmentation, Organ **PIXELS** in Zebrafish Embryo Images



More Examples, Satellite Images, Tracking Moving Objects in Videos

- https://www.mathworks.com/help/images/multispectral-semantic-segmentation-using-deep-learning.html
- https://www.mathworks.com/help/fusion/ug/visual-tracking-of-occluded-and-unresolved-objects.html
- https://www.mathworks.com/help/vision/ug/motion-based-multiple-object-tracking.html





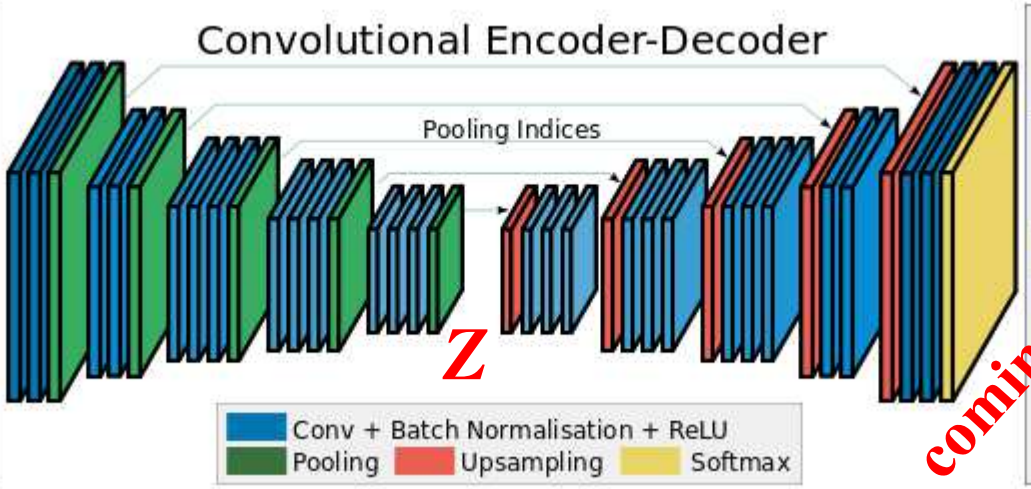
1:25

https://www.youtube.com/watch?app=desktop&v=RticXr3pWbQ

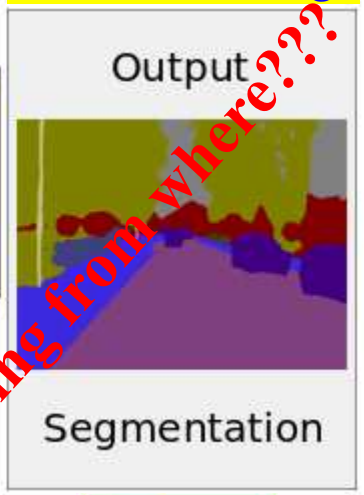
Semantic Segmentation Basic Idea: ***Encode + Decode*** CNNs

- It classifies **EACH** **pixel** into classes for object identification
- Semantic segmentation major steps➔ (still training together as **1** CNN)
  1. **Downsample** an image w/ ***Conv*** & ***Pooling***
  2. Then **upsample** the encoding (**Z-code**) to match the (pixel) ***labeled*** **image**
     - Upsampling = **tranposed** convolution layer ("**deconv**" or "**deconvolution**")

X = raw image

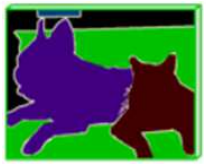


Y = labeled image



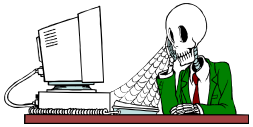
coming from where???

https://arxiv.org/pdf/1511.00561.pdf

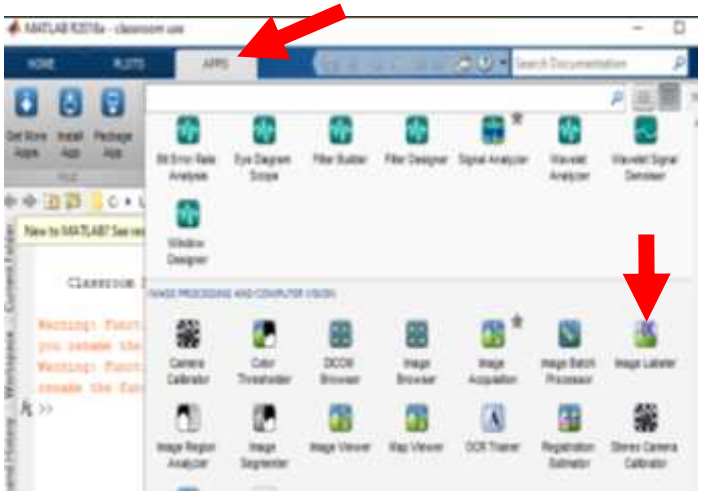


First, Labeling **Pixels** for Training Segmentation

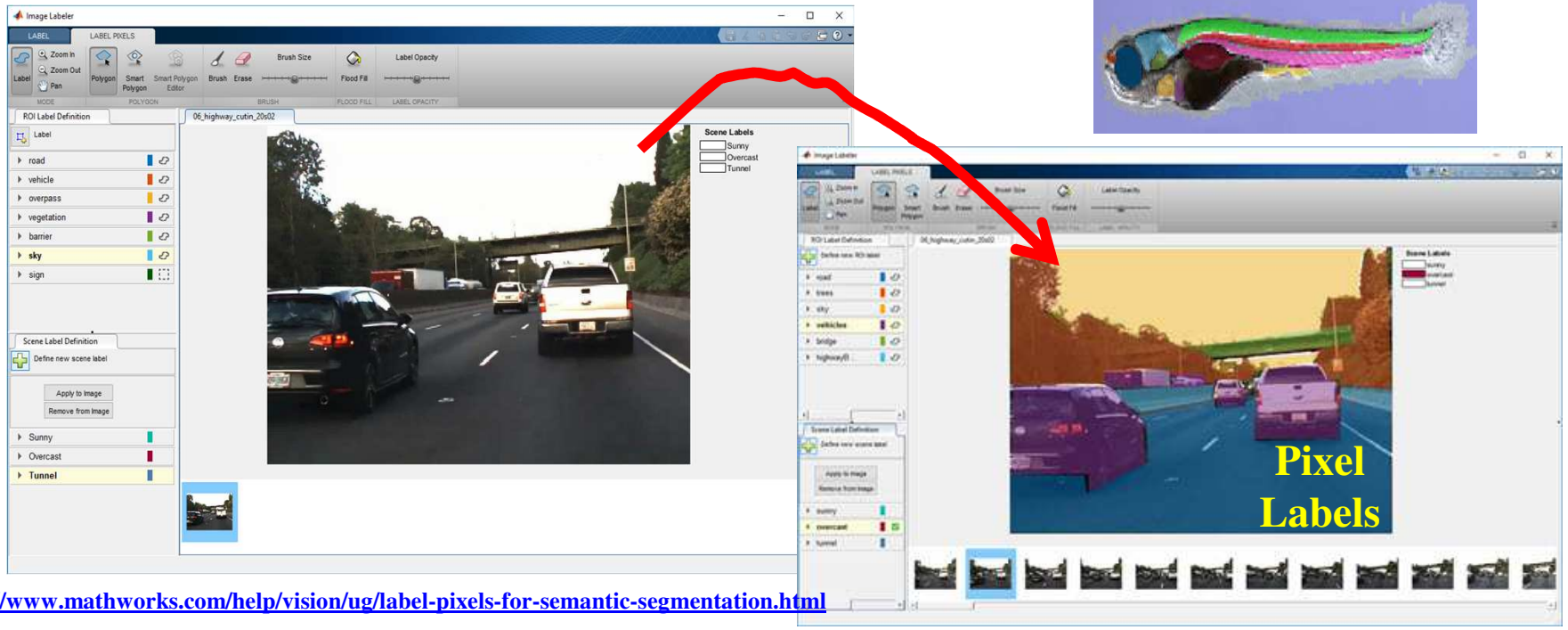
- A **very** time consuming process



- A Matlab tool to label pixels: **Image Labeler**



- **VGG Image Annotator** or **LabelMe**

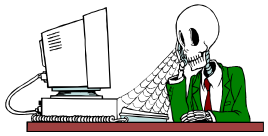


https://www.mathworks.com/help/vision/ug/label-pixels-for-semantic-segmentation.html

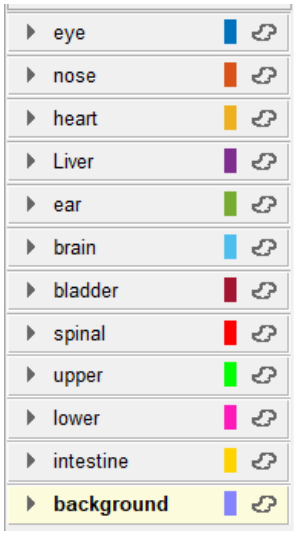
Image Annotator

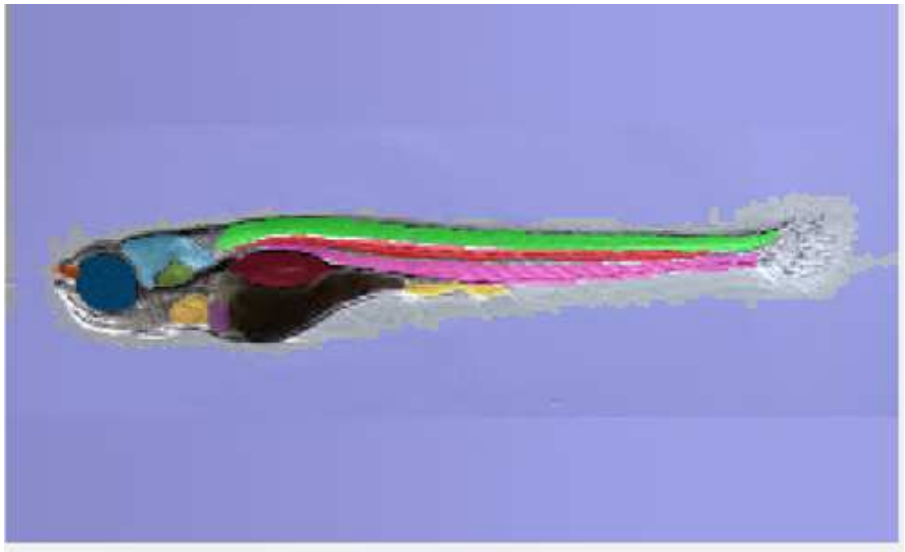
- **VGG Image Annotator**
- http://www.robots.ox.ac.uk/~vgg/software/via/

- **LabelMe**
- https://labelstud.io/label-studio-oss/?utm_adgroup=OSS-Data-Labeling&utm_source=google&utm_medium=paidsearch&utm_campaign=Label-Studio-Generic-Open-Source-97F&utm_term=data%20labeling%20tool%20open%20source&hsa_acc=5127568629&hsa_cam=21954076257&hsa_grp=170637749265&hsa_ad=723372299692&hsa_src=g&hsa_tgt=kwd-1006942380957&hsa_kw=data%20labeling%20tool%20open%20source&hsa_mt=b&hsa_net=adwords&hsa_ver=3&gad_source=1&gad_campaignid=21954076257&gbraid=0AAAAAo5UffItt3zwIEg4ZHh2it6lDGF_I&gclid=Cj0KCQjw58PGBhCkARIsADbDilx2KA5W7yUfjl822MyGTWD-eUWy_DJpITcxtG9l-8xxOPlEtl6-2GkaAlgREALw_wcB



- Matlab **Image Labeler**

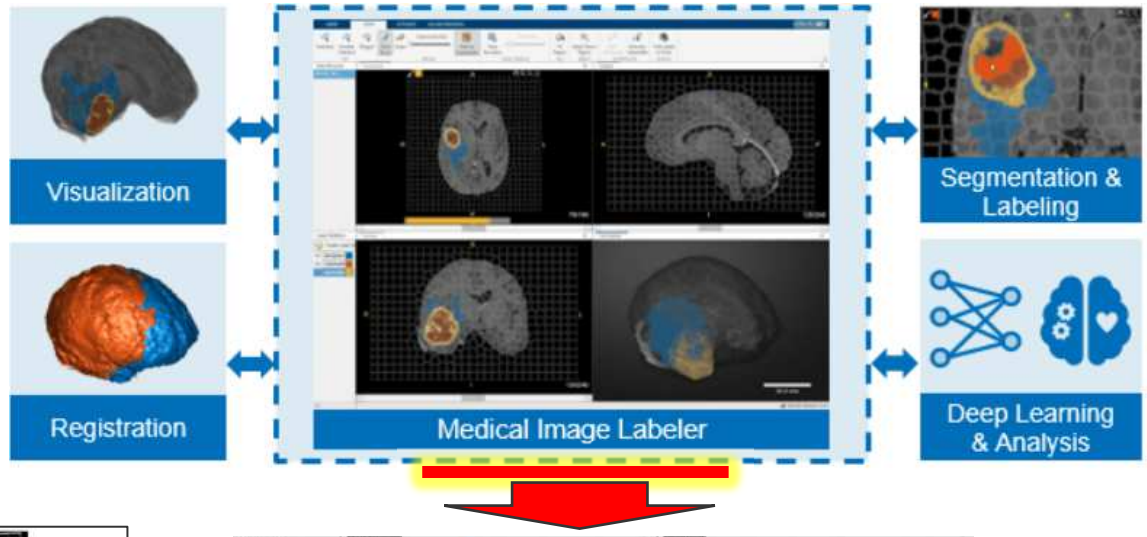


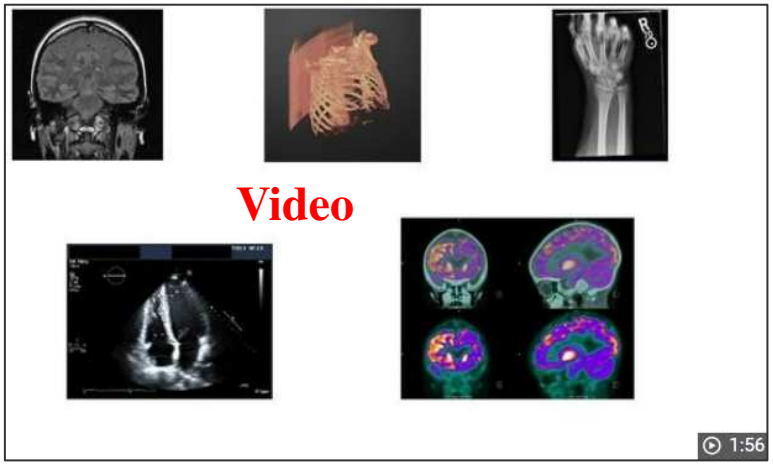


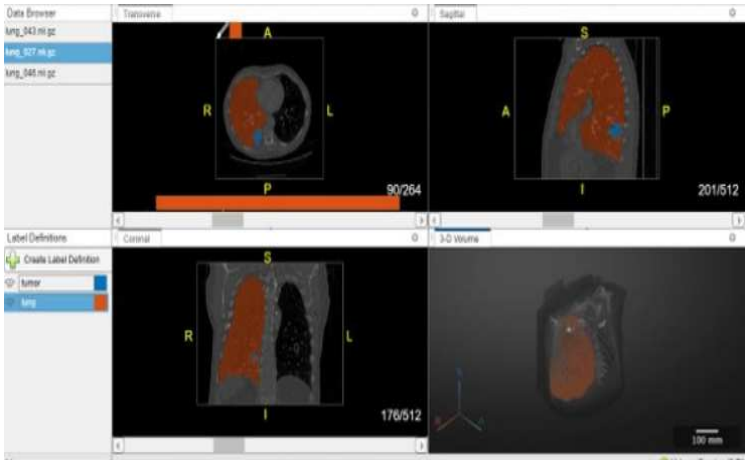
Matlab Medical Imaging Toolbox

- Medical Imaging Toolbox, visualize, register, segment, label 2D & 3D medical imgs
  - https://www.mathworks.com/products/medical-imaging.html
  - https://www.mathworks.com/help/medical-imaging/
  - https://www.mathworks.com/help/medical-imaging/ug/get-started-with-medical-image-labeler.html

F.Y.I.







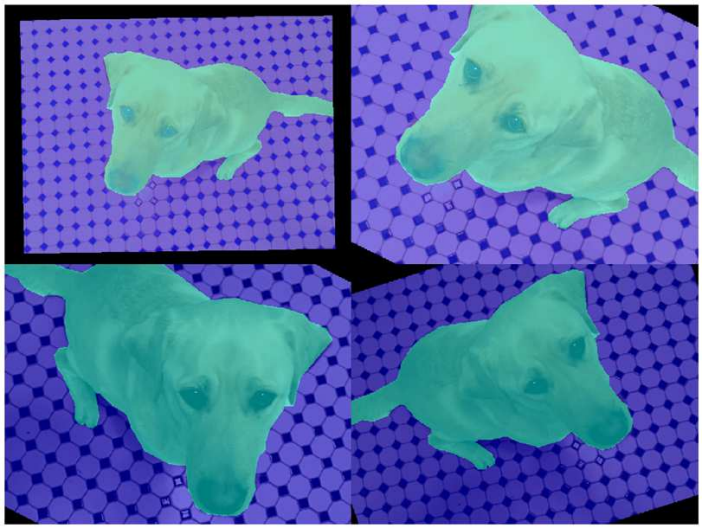
Raw Images ***AND*** *Label* Images in Augmentation

- Augmentation **MUST** apply **identical** transformations to **BOTH**…
  - **Input images** and **their labeled images**
  - i.e. rotate an input image & its labeled image w/ **same** degree
- **Except……**

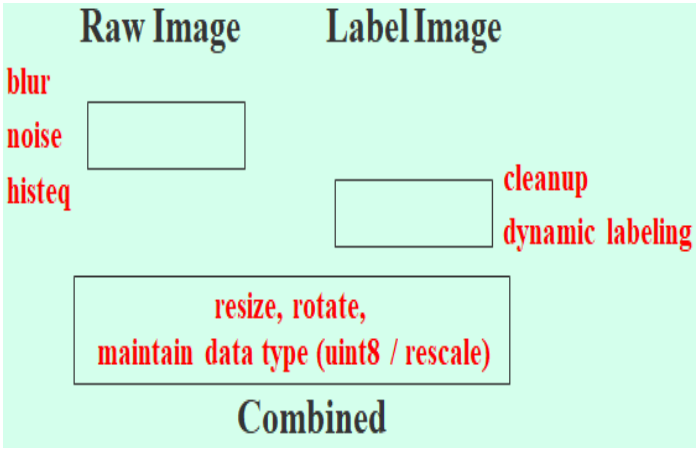
Other Pitfalls?

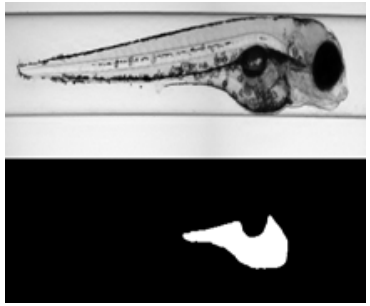
raw image

label image



HOW? Pitfalls?





raw

label

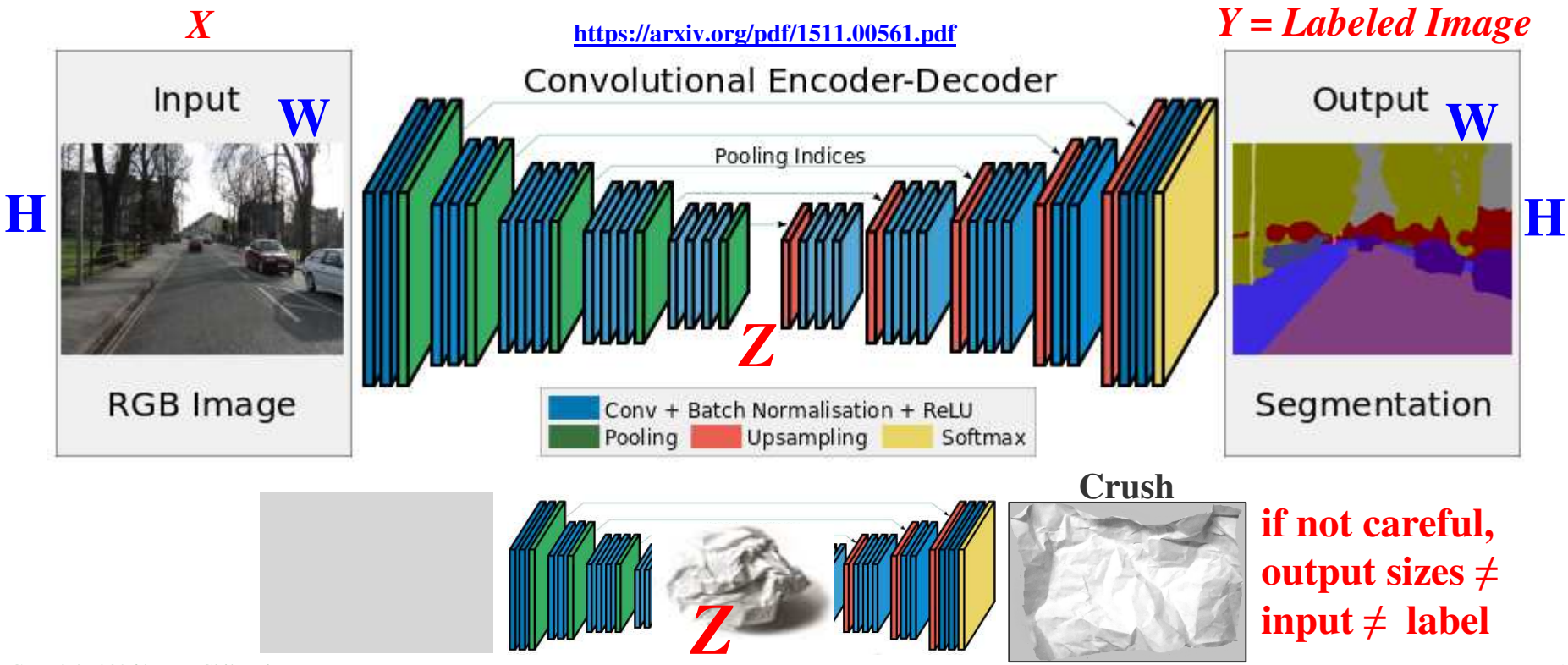
https://www.mathworks.com/help/deeplearning/ug/augment-pixel-labels-for-semantic-segmentation.html

Outline

- Object Detection, Fast RCNN
- Object Detection, Semantic Segmentation, Labeling, Applications
- √Encode + Decode CNN, Transposed Convolution, Pre-Trained CNN
- Quality Evaluation of Object Detection
- Identifying Morphological Changes in Organs w/ Occlusion Map

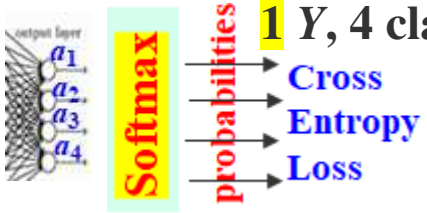
Semantic Segmentation (SegNet) using **Encode + Decode** CNN

- Semantic segmentation classifies **EACH** **pixel** into a class
  1. **Downsample** an image w/ layers of Conv & ReLU to ***Z*-code**
  2. THEN, **upsample** the encode (Z-code) to match / **predict** the **labeled** image
     - Upsampling w/ the **tranposed** convolution layer ("**deconv**" or "**deconvolution**" layer)
     - Transposed convolution highly influences the output **size** of the predicted label image



Last Decoder Layer = Pixel Classification / Softmax

- Last layer of decoder classifies **EACH** pixel into a class
  - 300 ×300 = 90,000 softmax + cross entropy on each pixel against **ALL** classes
  - Reason that semantic segmentation takes much longer than whole image classification



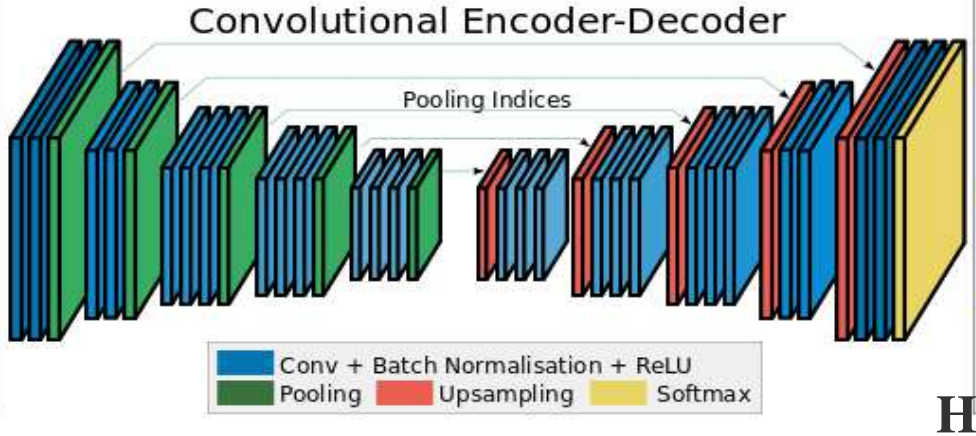
1 Y, 4 classes

H×W×#C

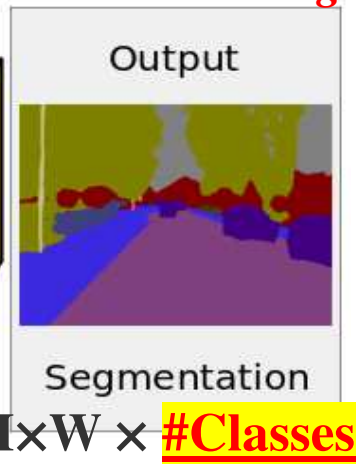
$$Softmax = P_i = \frac{e^{W^T X_i}}{\sum_i^m e^{W^T X_i}} = \frac{e^a}{sum(e^a)}$$

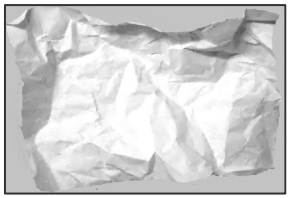
X

https://arxiv.org/pdf/1511.00561.pdf



Y = Labeled Image

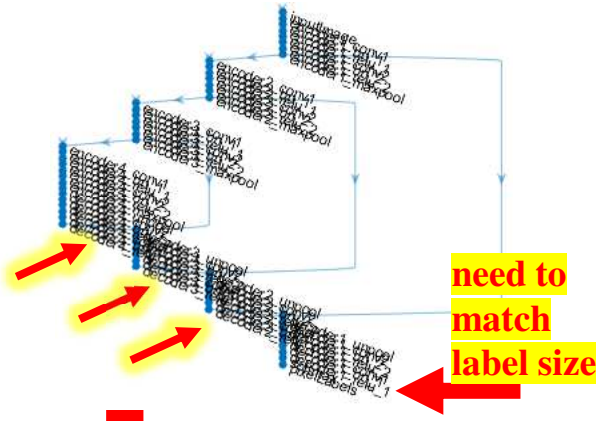




if not careful, output sizes ≠ input ≠ label

GUI to Find Output Size at **Each** Layer

- **net** = **alexnet**; **analyzeNetwork**(**net**)
- https://www.mathworks.com/help/deeplearning/ref/alexnet.html



Matlab GUI

net

Analysis date: 13-Jul-2018 13:57:10

25 layers 0 warnings 0 errors

12 layers 0 warnings 4 errors

ISSUES

	FOUND IN	MESSAGE
!	output	Missing softmax layer. A classification layer must be preceded by a softmax layer.
!	skipConv	Disconnected layers. All layers in the layer graph must be connected. Detected disconnected layers: layer 'skipConv'
!	add2	Missing input. Each layer input must be connected to the output of another layer. Detected missing inputs: input 'in3'
!	add1	Input size mismatch. Size of input to this layer is different from the expected input size. Inputs to this layer: from layer 'relu_2' (16×16×16 output) from layer 'relu_1' (32×32×16 output)

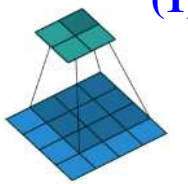
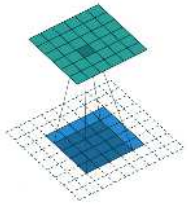
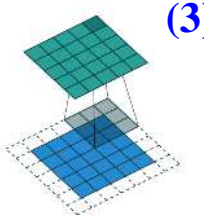
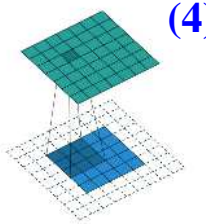
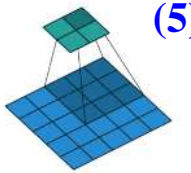
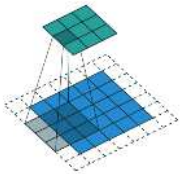
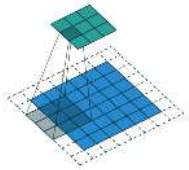
ANALYSIS RESULT

	NAME	TYPE	ACTIVATIONS	LEARNABLES
1	input 32x32x3 images with 'zerocenter' normalization	Image Input	32×32×3	-
2	conv_1 16 5x5x3 convolutions with stride [1 1] and padding 'same'	Convolution	32×32×16	Weights 5×5×3×16 Bias 1×1×16
3	relu_1 ReLU	ReLU	32×32×16	-
4	conv_2 16 3x3x16 convolutions with stride [2 2] and padding 'same'	Convolution	16×16×16	Weights 3×3×16×16 Bias 1×1×16
5	relu_2 ReLU	ReLU	16×16×16	-
6	! add1 Element-wise addition of 2 inputs	Addition	Unknown	-
7	conv_3 16 3x3x0 convolutions with stride [2 2] and padding 'same'	Convolution	Unknown	Weights Bias
8	relu_3 ReLU	ReLU	Unknown	-
9	! add2 Element-wise addition of 3 inputs	Addition	Unknown	-
10	fc 10 fully connected layer	Fully Connected	1×1×10	Weights Unknown Bias 10×1
11	relu6	ReLU	1×1×4096	-

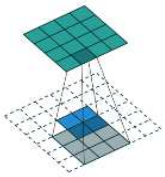
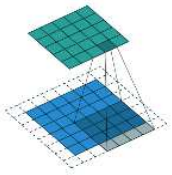
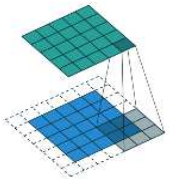
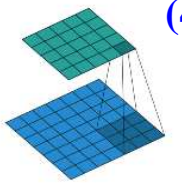
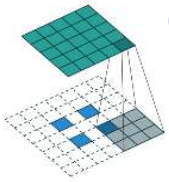
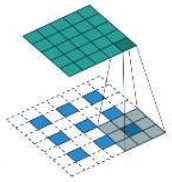
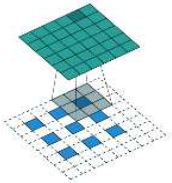
Transposed Convolution (De-Convolution)

- **Blue** = input, **green** = output, **gray area** = filter

Convolution

(1)	(2)	(3)	(4)
			
No padding, no strides	Arbitrary padding, no strides	Half padding, no strides	Full padding, no strides
(5)			**trainable downsizing convolution**
			
No padding, strides	Padding, strides	Padding, strides (odd)	

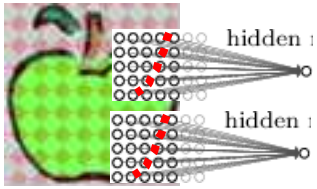
De-Convolution

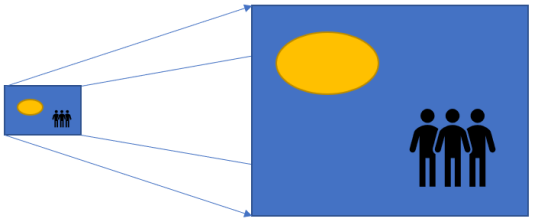
(1)	(2)	(3)	(4)
			
No padding, no strides, transposed	Arbitrary padding, no strides, transposed	Half padding, no strides, transposed	Full padding, no strides, transposed
(5)			**trainable upsizing de-convolution**
			
No padding, strides, transposed	Padding, strides, transposed	Padding, strides, transposed (odd)	

Conv output size

$$C = \left\lfloor \frac{W-F+2P}{S} \right\rfloor + 1$$

W = input width
F = filter size
P = padding size
S = striding size





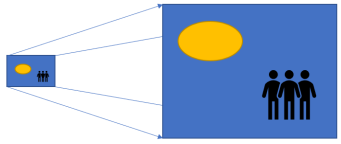
depends on *padding*, etc.

De-Conv output size??

https://github.com/vdumoulin/conv_arithmetic

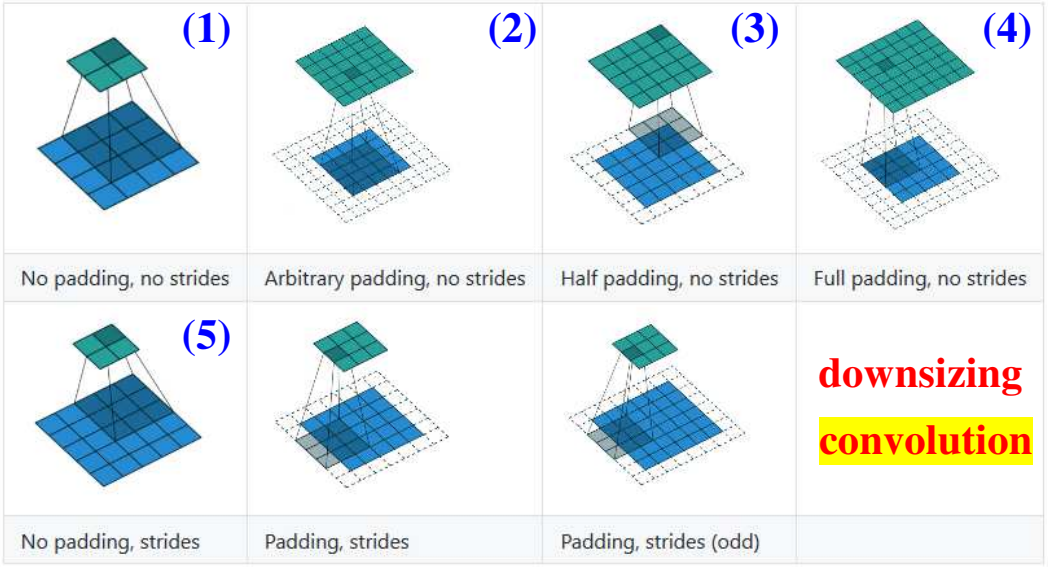
Transposed Convolution

(De-Convolution)

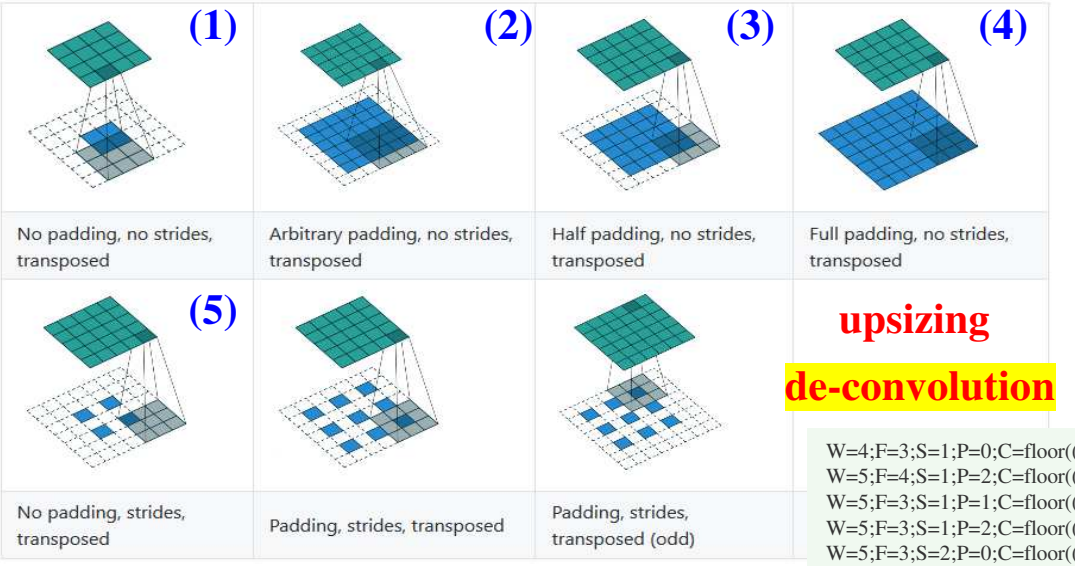


- **Blue** = input, **green** = output, **gray area** = filter

Convolution



De-Convolution



F.Y.I.

Conv output size

$$C = \left\lfloor \frac{W-F+2P}{S} \right\rfloor + 1$$

W = input width
F = filter size
P = padding size
S = striding size

=

De-Conv output size

$$D \approx (C-1) \times S + F - 2P$$

W=4;F=3;S=1; **P=0**;
C=floor((W-F+2*P)/S)+1; **D**=S*(C-1)+F-(2*P);
dP=floor(((W-1)*S-C+F)/2); **% (1) = 2 = #paddings**

W=5;F=4;S=1; **P=2**;
C=floor((W-F+2*P)/S)+1; **D**=S*(C-1)+F-(2*P);
dP=floor(((W-1)*S-C+F)/2); **% (2) = 1 = #paddings**

W=5;F=3;S=1; **P=1;**
C=floor((W-F+2*P)/S)+1; **D**=S*(C-1)+F-(2*P);
dP=floor(((W-1)*S-C+F)/2); **% (3) = 1 = #paddings**

W=5;F=3;S=1;P=2;
C=floor((W-F+2*P)/S)+1; **D**=S*(C-1)+F-(2*P);
dP=floor(((W-1)*S-C+F)/2); **% (4) = 0 = #paddings**

W=5;F=3;S=2;P=0;
C=floor((W-F+2*P)/S)+1; **D**=S*(C-1)+F-(2*P);
dP=floor(((W-1)*S-C+F)/2); **% (5) = 4**

```
W=4;F=3;S=1;P=0;C=floor((W-F+2*P)/S)+1; D=S*(C-1)+F-(2*P); dP=floor(((W-1)*S-C+F)/2);disp([W, C, D, dP])
W=5;F=4;S=1;P=2;C=floor((W-F+2*P)/S)+1; D=S*(C-1)+F-(2*P); dP=floor(((W-1)*S-C+F)/2);disp([W, C, D, dP])
W=5;F=3;S=1;P=1;C=floor((W-F+2*P)/S)+1; D=S*(C-1)+F-(2*P); dP=floor(((W-1)*S-C+F)/2);disp([W, C, D, dP])
W=5;F=3;S=1;P=2;C=floor((W-F+2*P)/S)+1; D=S*(C-1)+F-(2*P); dP=floor(((W-1)*S-C+F)/2);disp([W, C, D, dP])
W=5;F=3;S=2;P=0;C=floor((W-F+2*P)/S)+1; D=S*(C-1)+F-(2*P); dP=floor(((W-1)*S-C+F)/2);disp([W, C, D, dP])
```

 https://github.com/vdumoulin/conv_arithmetic

Examples of *Changing* De-Conv Output Sizes

$$C = \left\lfloor \frac{W-F+2P}{S} \right\rfloor + 1$$

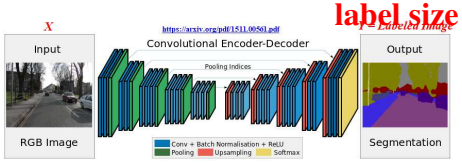
- **Convolution** (down sampling)

W = input width

C_{out_size}

- W = 28; F = 5; P = 0; S = 1; **conv_size** = floor((W - F + 2 * P) / S) + 1 = 24
- W = 28; F = 5; P = 0; S = 2; **conv_size** = floor((W - F + 2 * P) / S) + 1 = 12
- W = 28; F = 5; P = 2; S = 2; **conv_size** = floor((W - F + 2 * P) / S) + 1 = 14
- W = 28; F = 4; P = 0; S = 2; **conv_size** = floor((W - F + 2 * P) / S) + 1 = 13

need to match label size



- **Transpose Convolution** (up sampling)
  - C = 24; F = 5; P = 0; S = 1; **de_conv_size** = (C - 1) * S + F – 2 * P = 28
  - C = 12; F = 5; P = 0; S = 2; **de_conv_size** = (C - 1) * S + F – 2 * P = **27**
  - C = 14; F = 5; P = 2; S = 2; **de_conv_size** = (C - 1) * S + F - 2 * P = **27**
  - C = 13; F = 4; P = 0; S = 2; **de_conv_size** = (C - 1) * S + F – 2 * P = 28

$$W \approx (C-1) \times S + F - 2P$$

W = input width
F = filter size
P = padding size
S = striding size
C = conv size

- **Solution 2**➔ produce bigger images then **Cropping** 2016
  - transposedConv2dLayer(FSize, #Filters, 'Stride', 2, 'Cropping', 2);
    - https://www.mathworks.com/help/nnet/ref/transposedconv2dlayer.html
  - model.add(Cropping2D(cropping=((2, 2), (2, 2))))
    - https://keras.io/layers/convolutional/
- Case 2: C = 12; F = 5; P = **4**; S = **3**; **de_conv_size** = (C - 1) * S + F – 2 * P = **30**
- Case 2: C = 12; F = 5; P = **5**; S = **3**; **de_conv_size** = (C - 1) * S + F – 2 * P = **28**

Solution 1➔ **estimate padding in de-conv layer**

$$dP \approx \left\lfloor \frac{(w-1) \times S - C + F}{2} \right\rfloor$$

Solution 3???

Resize Layer or Xfer
2021--2021

Why Care??? Can You Convert Output Back to 64 × 64 × 3?

- **VAI_Slide_WHY_Care_Cases\Why_Care_CNN_Down_Up_Sample_Size.m**
  Why_Care_CNN_Down_Up_Sample_Size_ANSWER.m
  Why_Care_CNN_Down_Up_Sample_Size_ANSWER.ipynb

	Name	Type	Activations	Learnable Properties
1	imageinput 64×64×3 images with 'zerocenter' normalization	Image Input	64(S) × 64(S) × 3(C) × 1(B)	-
2	conv_1 8 6×6 convolutions with stride [1 1] ar	2-D Convolution	61(S) × 61(S) × 8(C) × 1(B)	Weights 6 × 6 × 3 × 8
3	maxpool_1 2×2 max pooling with stride [2 2] and			
4	conv_2 16 4×4 convolutions with stride [1 1] a			
5	maxpool_2 2×2 max pooling with stride [2 2] and			
6	conv_3 32 3×3 convolutions with stride [1 1] a			
7	maxpool_3 2×2 max pooling with stride [2 2] and			
8	transposed-conv_1 32 3×3 transposed convolutions with			
9	transposed-conv_2 32 3×3 transposed convolutions with			
10	transposed-conv_3 16 4×4 transposed convolutions with			

≠

	Name	Type	Activations	Learnable Sizes
1	imageinput 64×64×3 images with 'zerocenter' norma...	Image Input	64(S) × 64(S) × 3(C) × 1(B)	-
2	conv_1 8 6×6 convolutions with stride [1 1] and ...	2-D Convolution	61(S) × 61(S) × 8(C) × 1(B)	Weig… 6 × 6 × 3 … Bias 1 × 1 × 8
3	maxpool_1 2×2 max pooling with stride [2 2] and pa...	2-D Max Pooling	30(S) × 30(S) × 8(C) × 1(B)	-
4	conv_2 16 4×4 convolutions with stride [1 1] and...	2-D Convolution	27(S) × 27(S) × 16(C) × 1(B)	Weig… 4 × 4 × 8 … Bias 1 × 1 × 16
5	maxpool_2 2×2 max pooling with stride [2 2] and pa...	2-D Max Pooling	13(S) × 13(S) × 16(C) × 1(B)	-
6	conv_3 32 3×3 convolutions with stride [1 1] and...	2-D Convolution	13(S) × 13(S) × 32(C) × 1(B)	Weig… 3 × 3 × 16… Bias 1 × 1 × 32
7	maxpool_3 2×2 max pooling with stride [2 2] and pa...	2-D Max Pooling	6(S) × 6(S) × 32(C) × 1(B)	-
8	transposed-conv_1 32 3×3 transposed convolutions with stri...	2-D Transposed Con...	13(S) × 13(S) × 32(C) × 1(B)	Weig… 3 × 3 × 32… Bias 1 × 1 × 32
9	transposed-conv_2 32 3×3 transposed convolutions with stri...	2-D Transposed Con...	27(S) × 27(S) × 32(C) × 1(B)	Weig… 3 × 3 × 32… Bias 1 × 1 × 32
10	transposed-conv_3 16 4×4 transposed convolutions with stri...	2-D Transposed Con...	56(S) × 56(S) × 16(C) × 1(B)	Weig… 4 × 4 × 16… Bias 1 × 1 × 16
11	layer Resize 2d layer with output size of [64 64].	Resize	64(S) × 64(S) × 16(C) × 1(B)	-

Solution 3

Convert Output Back to
$64 \times 64 \times 3$

Layer (type)	Output Shape
conv2d (Conv2D)	(None, 64, 64, 8)
max_pooling2d (MaxPooling2D)	(None, 32, 32, 8)
conv2d_1 (Conv2D)	(None, 29, 29, 16)
max_pooling2d_1 (MaxPooling2D)	(None, 14, 14, 16)
conv2d_2 (Conv2D)	(None, 14, 14, 32)
max_pooling2d_2 (MaxPooling2D)	(None, 7, 7, 32)
conv2d_transpose (Conv2DTranspose)	(None, 14, 14, 32)
conv2d_transpose_1 (Conv2DTranspose)	(None, 28, 28, 32)
conv2d_transpose_2 (Conv2DTranspose)	(None, 56, 56, 3)
resizing (Resizing)	(None, 64, 64, 3)

Why_Care_CNN_Down_Up_Sample_Size_ANSWER.ipynb

```
# input
keras.Input(shape=(64, 64, 3)),
# --- downsampling ---
layers.Conv2D(8, kernel_size=6, padding="same"),
layers.MaxPool2D(pool_size=2, strides=2),

layers.Conv2D(16, kernel_size=4, padding="valid"),
layers.MaxPool2D(pool_size=2, strides=2),

layers.Conv2D(32, kernel_size=3, padding="same"),
layers.MaxPool2D(pool_size=2, strides=2),

# --- upsampling (transpose conv) ---
layers.Conv2DTranspose(32, kernel_size=3, strides=2, padding="same")
layers.Conv2DTranspose(32, kernel_size=3, strides=2, padding="same")
layers.Conv2DTranspose(3,  kernel_size=4, strides=2, padding="same")
layers.Resizing(64, 64),          # --- resize to 64x64
```

Name	Type	Activations
imageinput 64×64×3 images with 'zerocenter' norma...	Image Input	64(S) × 64(S)
conv_1 8 6×6 convolutions with stride [1 1] and ...	2-D Convolution	61(S) × 61(S)
maxpool_1 2×2 max pooling with stride [2 2] and pa...	2-D Max Pooling	30(S) × 30(S)
conv_2 16 4×4 convolutions with stride [1 1] and...	2-D Convolution	27(S) × 27(S)
maxpool_2 2×2 max pooling with stride [2 2] and pa...	2-D Max Pooling	13(S) × 13(S)
conv_3 32 3×3 convolutions with stride [1 1] and...	2-D Convolution	13(S) × 13(S)
maxpool_3 2×2 max pooling with stride [2 2] and pa...	2-D Max Pooling	6(S) × 6(S) ×
transposed-conv_1 32 3×3 transposed convolutions with stri...	2-D Transposed Con...	13(S) × 13(S)
transposed-conv_2 32 3×3 transposed convolutions with stri...	2-D Transposed Con...	27(S) × 27(S)
transposed-conv_3 16 4×4 transposed convolutions with stri...	2-D Transposed Con...	56(S) × 56(S)
layer Resize 2d layer with output size of [64 64].	Resize	64(S) × 64(S)

Solution 3

Why Care Examples… with Visual Inspection

- **Folder**: VAI_Slide_WHY_Care_Cases

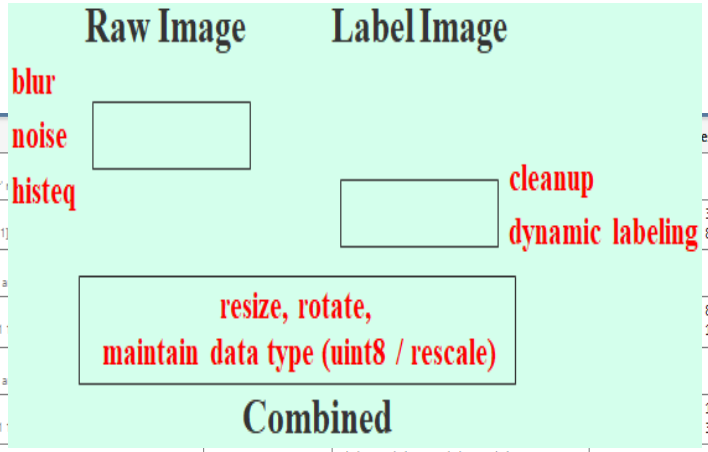
- Downsampling + Upsampling ➔ mismatched input and output sizes
  - Why_Care_CNN_Down_Up_Sample_Size.m
  - Why_Care_CNN_Down_Up_Sample_Size_ANSWER.m
  - **Why_Care_CNN_Down_Up_Sample_Size_ANSWER.ipynb**

- Bug in creating / editing / connecting NN
  - Why_Visual_Insepct_NN.m

- Changing input layer size
  - AlexNet_XFer_ErrorIssue_21S.m

- **To be discussed later:**
- **Augmentation problem**
  - **Why_Care_Color_Look_Nice_or_Not.m**
  - **Why_Care_Image_Normalization.m**

HOW? Pitfalls?

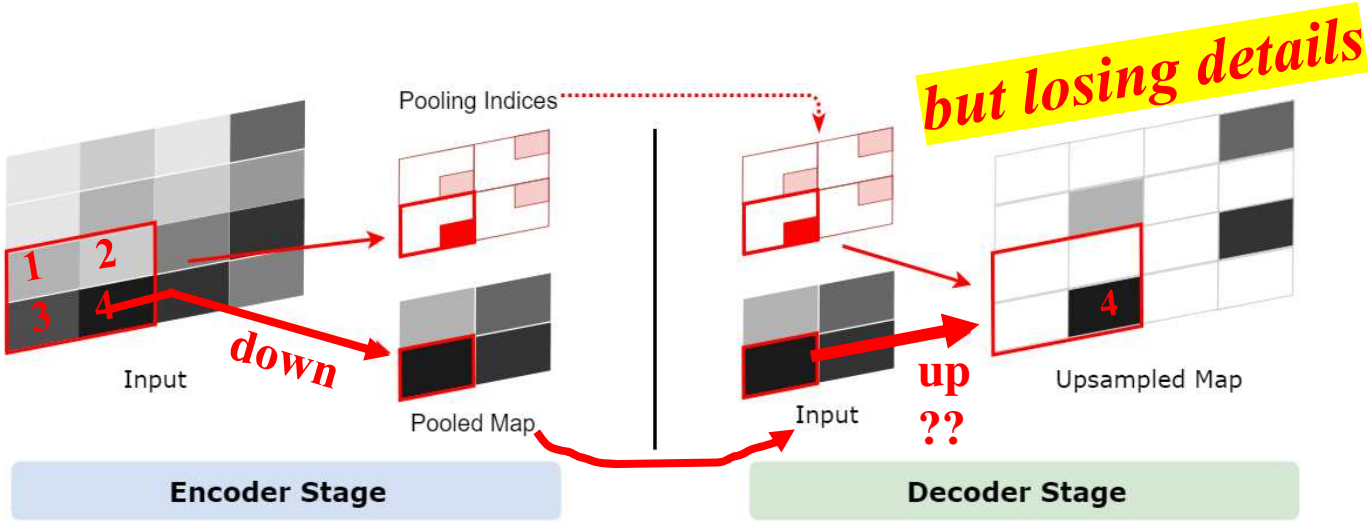


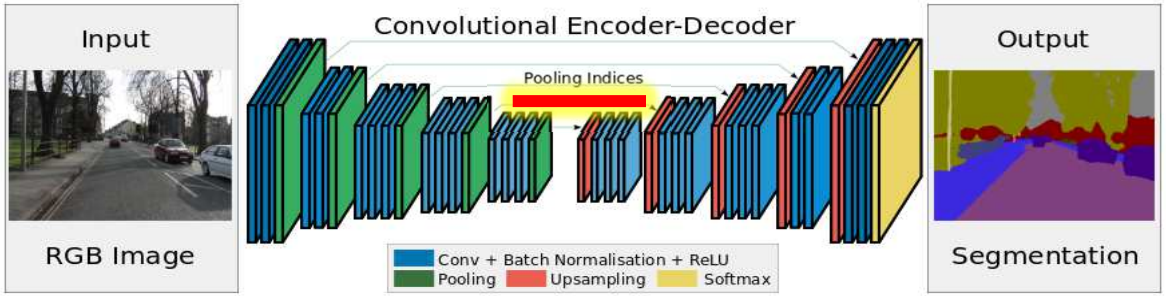
	Name			
1	imageinput 64×64×3 images with 'zerocenter'			
2	conv_1 8 6×6 convolutions with stride [1 1]			3 × 8 8
3	maxpool_1 2×2 max pooling with stride [2 2] a			
4	conv_2 16 4×4 convolutions with stride [1			8 × 16 16
5	maxpool_2 2×2 max pooling with stride [2 2] a			
6	conv_3 32 3×3 convolutions with stride [1			16 × 32 32
7	maxpool_3 2×2 max pooling with stride [2 2] and padding [0 0 0 0]	2-D Max Pooling	6(S) × 6(S) × 32(C) × 1(B)	-
8	transposed-conv_1 32 3×3 transposed convolutions with stride [2 2] and cropping [0 0 0 0]	2-D Transposed Con...	13(S) × 13(S) × 32(C) × 1(B)	Weights 3 × 3 × 32 × 32 Bias 1 × 1 × 32
9	transposed-conv_2 32 3×3 transposed convolutions with stride [2 2] and cropping [0 0 0 0]	2-D Transposed Con...	27(S) × 27(S) × 32(C) × 1(B)	Weights 3 × 3 × 32 × 32 Bias 1 × 1 × 32
10	transposed-conv_3 16 4×4 transposed convolutions with stride [2 2] and cropping [0 0 0 0]	2-D Transposed Con...	56(S) × 56(S) × 16(C) × 1(B)	Weights 4 × 4 × 16 × 32 Bias 1 × 1 × 16

Another Problem in Pooling / Upsampling ➔ Pooling Indices

- Upsampling = opposite of pooling… **BUT**… with **uncertainty** & **lost of details**

- Pre-trained **SegNet** uses ***Pooling Indices*** to address **(not solve)** uncertainty
  - Save the index (location) of max values during pooling for decoder to use



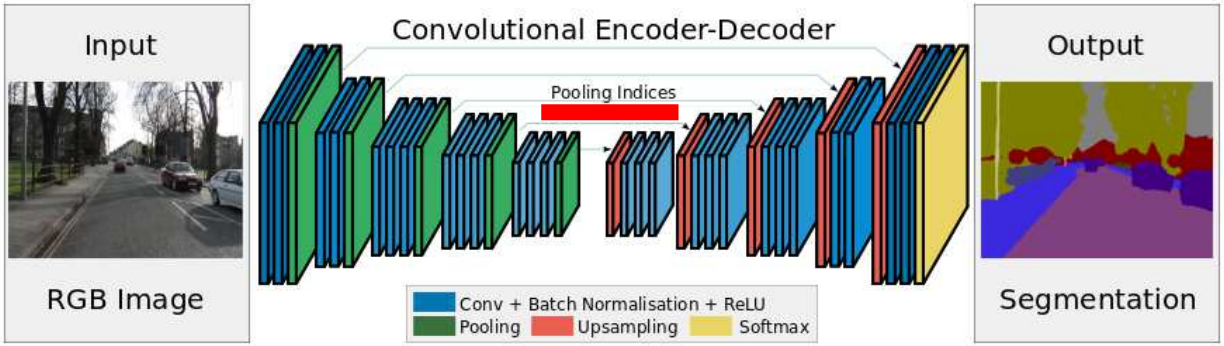


https://arxiv.org/pdf/1511.00561.pdf

Pooling, Upsampling & Pooling Indices, Another View

Figure : Max Unpooling

https://meetshah1995.github.io/semantic-segmentation/deep-learning/pytorch/visdom/2017/06/01/semantic-segmentation-over-the-years.html

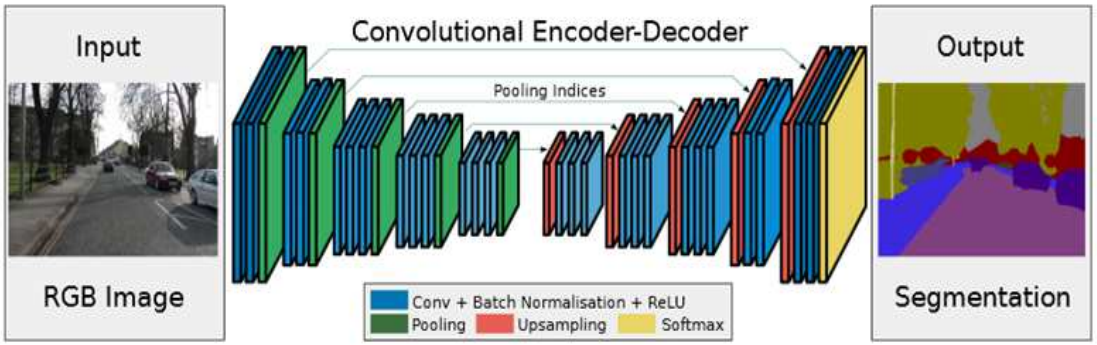


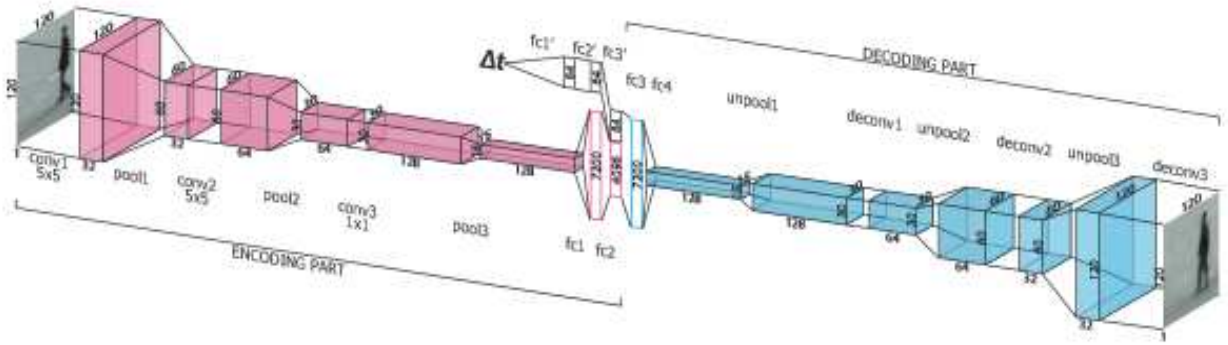
https://arxiv.org/pdf/1511.00561.pdf

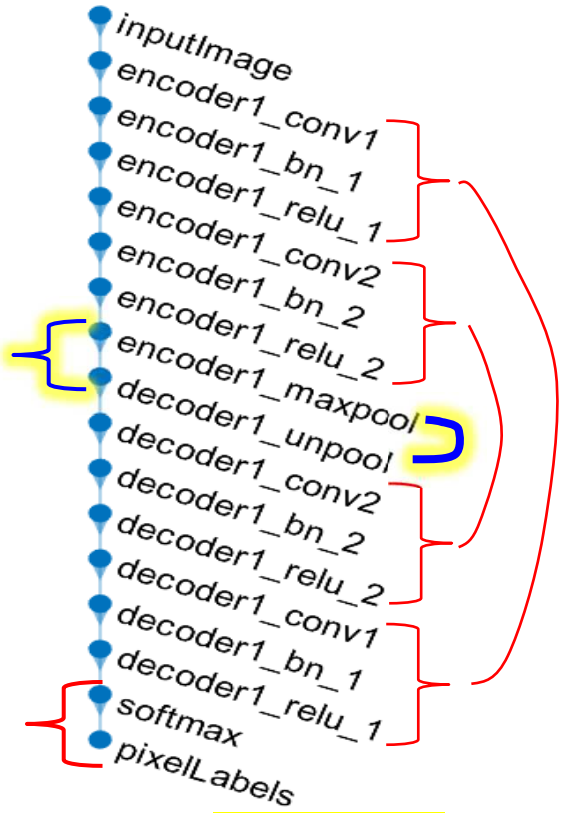
Close Look of Encoder/Decoder Layers

- Be careful about the **order** of layers…

- Encoder→ **conv**, BN, RELU, **Pooling**
- Decode→ **Uppooling**, **de-conv**, BN, RELU → **same**







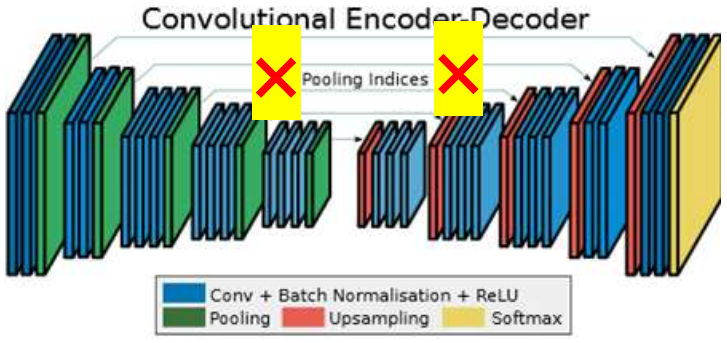
H×W × #Classes

Implement Your Own **Keras** Transposed Conv https://keras.io/layers/convolutional/

- Many examples at https://www.programcreek.com/python/example/89672/keras.layers.Conv2DTranspose
  - keras.layers.**Conv2DTranspose**(filters, kernel_size, strides=(1, 1), padding='valid', …)
  - keras.layers.**UpSampling2D**(size=(2, 2), data_format=None, interpolation='nearest')

```
self.G = Sequential()
# conv layers  <<<  HERE
.........
# de-conv layers
self.G.add(UpSampling2D())
self.G.add(Conv2DTranspose(int(depth/2), 5, padding='same'))
self.G.add(BatchNormalization(momentum=0.9))
self.G.add(Activation('relu'))
self.G.add(UpSampling2D())
self.G.add(Conv2DTranspose(int(depth/4), 5, padding='same'))
self.G.add(BatchNormalization(momentum=0.9))
self.G.add(Activation('relu'))
.........
```

```
layers = [
  %% downsampling
  imageInputLayer([32 32 1])
  convolution2dLayer([5 5], 32, 'Padding', 1);
  reluLayer()
  maxPooling2dLayer(2, 'Stride', 2)
  ......
  %% upsampling
  transposedConv2dLayer([5 5], 32, 'Stride', 2)
  ......
```

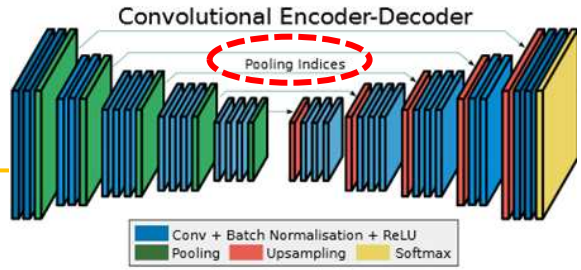


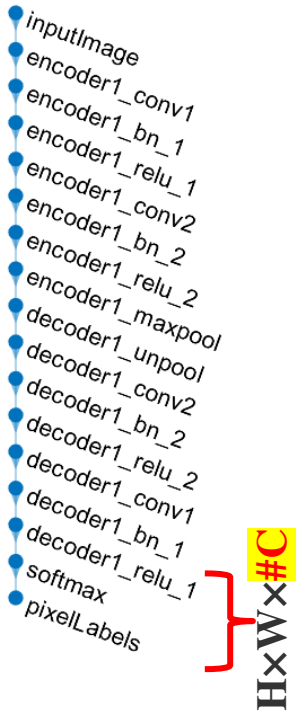
Transfer Learning

imageSize = [32 32]; **Class** = 2;

segnetLayers(Img_Size, **Class**, **#encoders**);

```
numFilters = 64;      filterSize = 3;      numClasses = 2;
layers = [
   %% downsampling
   imageInputLayer([32 32 1])
   convolution2dLayer(filterSize, numFilters, 'Padding', 1);  reluLayer()
   maxPooling2dLayer(2, 'Stride', 2)
   ......
   %% upsampling
   transposedConv2dLayer(4, numFilters, 'Stride', 2);
   ......
%% final layers
   convolution2dLayer(1, numClasses);
   softmaxLayer()
   pixelClassificationLayer()          ]
```





- Before Softmax **add** a conv using **1×1** convolution w/ **# of filters = #classes**
  - https://www.mathworks.com/help/vision/ug/semantic-segmentation-with-deep-learning.html#mw_6ab02754-d2fa-4330-8bea-3eeec77279da

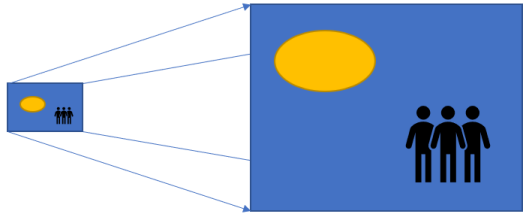
Outline

- Object Detection, Fast RCNN
- Object Detection, Semantic Segmentation, Labeling, Applications
- Encode + Decode CNN, Transposed Convolution, Pre-Trained CNN
  - Example applications
- Quality Evaluation of Object Detection
- Identifying Morphological Changes in Organs w/ Occlusion Map

Appendix

- Transposed Convolution (De-Convolution)

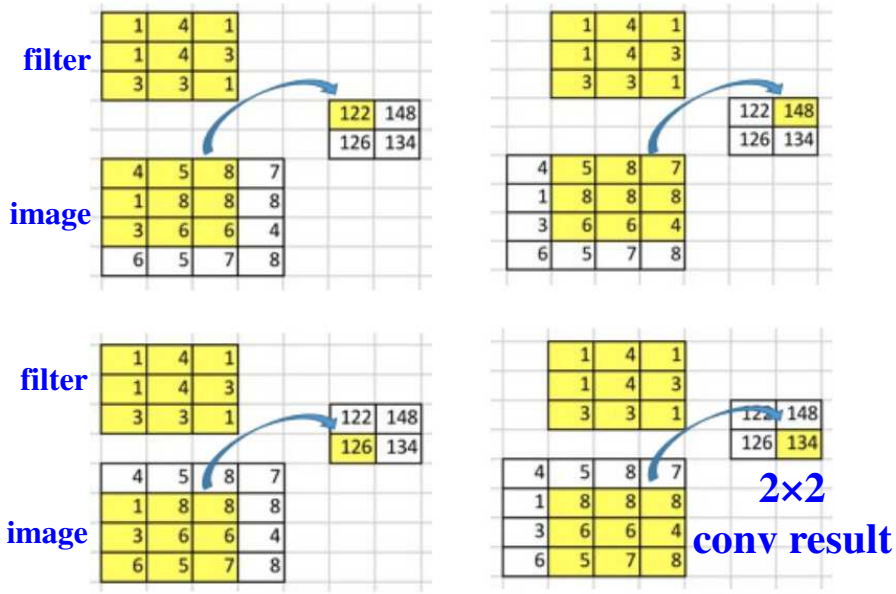
Transposed Convolution (De-Convolution)



- https://towardsdatascience.com/up-sampling-with-transposed-convolution-9ae4f2df52d0

F.Y.I.

conv of 4×4 image w/ 3×3 filter ➔ 2×2 (without padding)



change conv result from 2×2 to 4×1

organize 3×3 conv filter to 4×16 de-conv filter

(4×1)' * (4×16) = 1×16 = 4×4 deconv result

organize the original 3×3 conv filters into a 4×16 de-conv filter